**DEEP LEARNING FROM SCRATCH**

A Guide for Self-Learners

# Episode VII

*Deep*

*Two layers was the beginning.*
*Now we remove the limit.*

***Now we go deep.***

*One list. Any architecture. Infinite depth.*

***Why this episode?***

*Your 2-layer network works. But it's hardcoded.*
*What if you wanted 5 layers? Or 50?*
*In this episode, you'll generalize everything —*
*and discover that the code barely changes.*

*"Simplicity is the ultimate sophistication."*
*– Leonardo da Vinci*

**Pierre Chambet**

March 2026

---

Part of the *Deep Learning From Scratch* series — an independent reconstruction of AI.

# Contents



**What You'll Learn in This Episode**

After these pages, you'll be able to build a neural network of **any depth** — not just 2 layers, but $L$ layers — from scratch.

1. How a single **dimensions list** defines an entire architecture
2. How to **generalize initialization** with a loop
3. How to write **forward propagation** for $L$ layers
4. How to derive and implement **generalized backpropagation**
5. How to test your network on **increasingly complex datasets**
6. Why **deeper networks** don't always mean better results
7. Where the next frontier lies: **discipline over power**

# 1 What If?

In Episode VI, you built a neural network from scratch.

It worked. It classified toxic plants with near-perfect accuracy. It even handled real images — cats and dogs — with respectable results.

But here's the thing: that network was **hardcoded**.

Two layers. Exactly two. The initialization function created `W1`, `b1`, `W2`, `b2` — four parameters, by name, one by one. The forward propagation computed `Z1`, `A1`, `Z2`, `A2` — again, hardcoded. The backpropagation derived `dW1`, `db1`, `dW2`, `db2` — six lines, each written by hand.

What if you wanted three layers?

You'd have to rewrite the initialization. Rewrite forward propagation. Rewrite backpropagation. Add more variables. More lines. More chances for bugs.

What about five layers? Ten?

*That approach doesn't scale.*

**The Core Idea**

The operations at each layer are **identical**: multiply, add bias, activate. The only things that change are the dimensions.
If the operations are the same, we can use a **loop**.
And if we use a loop, we can handle **any number of layers**.

That's what this episode is about. We're going to take every function from Episode VI and replace the hardcoded indices with a loop. The result: a neural network of **any depth**, controlled by a single list.

**Let's Build**

Open the companion notebook. Every function we discuss here is ready to run.
**Colab:** https://colab.research.google.com/github/Pchambet/Deep-Learning-from-Scratch/blob/main/notebooks/07_deep.ipynb

**Where We Left Off**

Episode VI gave you 7 working functions:

- `initialisation(n0, n1, n2)` — hardcoded for 2 layers
- `forward_propagation(X, parameters)` — Z1, A1, Z2, A2
- `back_propagation(y, parameters, activations)` — 6 gradients
- `update(gradients, parameters, learning_rate)`
- `predict(X, parameters)`
- `neural_network(...)` — the training loop
- `decision_boundary(...)` — visualization

Today, we generalize all of them to $L$ layers.

## 2 The Architecture List

In Episode VI, the architecture was defined by three numbers: `n0` (input dimension), `n1` (hidden neurons), and `n2` (output). You passed them as separate arguments.

Now, we replace them with a single list:

```
dimensions = [2, 32, 32, 1]
```

This single line says:

- **Input layer**: 2 features
- **Hidden layer 1**: 32 neurons
- **Hidden layer 2**: 32 neurons
- **Output layer**: 1 neuron (binary classification)

The number of layers is:

$$L = \texttt{len(dimensions)} - 1$$

For our example: $L = 4 - 1 = 3$ layers (two hidden + one output).

**The Dimensions List**

A neural network architecture is fully defined by:

$$\texttt{dimensions} = [n_0,\ n_1,\ n_2,\ \ldots,\ n_L]$$

where $n_0$ is the input dimension, $n_l$ is the number of neurons in layer $l$, and $n_L$ is the output dimension.

**One list. Any architecture. Any depth.**

Want a deeper network? Just add numbers:

```
dimensions = [4096, 64, 32, 16, 1]   # 4 layers
dimensions = [4096, 128, 128, 64, 32, 16, 1]   # 6 layers
```

No code changes. Just a longer list.

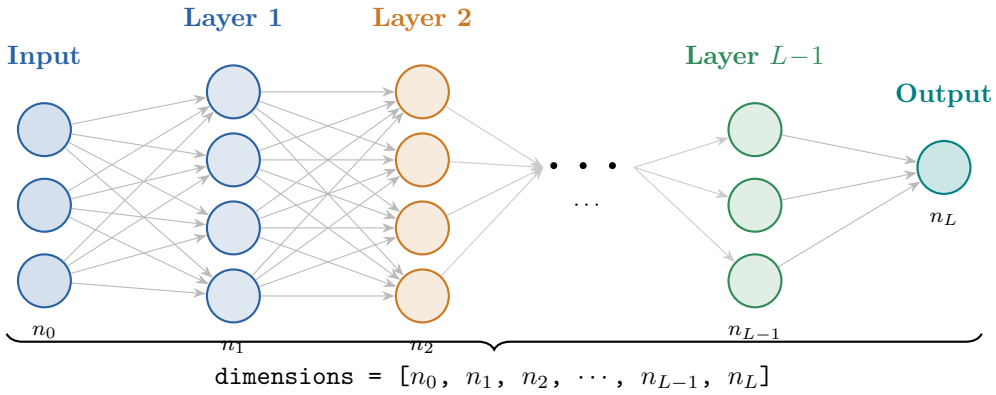


Figure 1: A generic $L$-layer neural network. The entire architecture is defined by the `dimensions` list.

## 3 Initialization — The Loop

Let's start with the comparison. Here's what initialization looked like in Episode VI:

Episode VI — Hardcoded for 2 layers

```
def initialisation(n0, n1, n2):
    parameters = {
        'W1': np.random.randn(n1, n0),
        'b1': np.random.randn(n1, 1),
        'W2': np.random.randn(n2, n1),
        'b2': np.random.randn(n2, 1),
    }
    return parameters
```

Four lines, four parameters, two layers. Hardcoded.
Now here's the generalized version:

Episode VII — Any number of layers

```
def initialisation(dimensions):
    parameters = {}
    L = len(dimensions)
    for l in range(1, L):
        parameters[f'W{l}'] = np.random.randn(dimensions[l],
                                              dimensions[l-1])
        parameters[f'b{l}'] = np.random.randn(dimensions[l], 1)
    return parameters
```

That's it. The loop creates `W1`, `b1`, `W2`, `b2`, ..., `WL`, `bL` — however many layers you specify.

**Same Logic, Different Scale**

If you call `initialisation([2, 32, 32, 1])`, the loop creates:

- $l = 1$: `W1` shape $(32, 2)$, `b1` shape $(32, 1)$
- $l = 2$: `W2` shape $(32, 32)$, `b2` shape $(32, 1)$
- $l = 3$: `W3` shape $(1, 32)$, `b3` shape $(1, 1)$

Exactly what Episode VI had — plus one extra layer. Same logic. Just a loop.

Let's verify it works:

```
parameters = initialisation([2, 32, 32, 1])
for key, value in parameters.items():
    print(key, value.shape)
```

**Output**

```
W1 (32, 2)
b1 (32, 1)
W2 (32, 32)
b2 (32, 1)
W3 (1, 32)
b3 (1, 1)
```

Six parameters instead of four. Three layers instead of two. **Same function.**

## 4 Forward Propagation — The Generic Loop

In Episode VI, forward propagation was four lines:

Episode VI — Hardcoded

```
Z1 = W1 @ X + b1      # A1 = sigma(Z1)
A1 = 1 / (1 + np.exp(-Z1))
Z2 = W2 @ A1 + b2     # A2 = sigma(Z2)
A2 = 1 / (1 + np.exp(-Z2))
```

Look at the pattern: for each layer, we compute $Z = W \cdot A_{\text{prev}} + b$ and then $A = \sigma(Z)$. The generalized version replaces this with a loop:

Episode VII — Any depth

```
def forward_propagation(X, parameters):
    activations = {'A0': X}
    L = len(parameters) // 2
    A = X
    for l in range(1, L + 1):
        Z = parameters[f'W{l}'] @ A + parameters[f'b{l}']
        A = 1 / (1 + np.exp(-Z))
```

```
            activations[f'A{l}'] = A
    return activations
```

Let's break it down:

1. We set the input $X$ as the "zeroth activation": $A_0 = X$.
2. For each layer $l$ from 1 to $L$:
   - Compute the pre-activation: $Z_l = W_l \cdot A_{l-1} + b_l$
   - Apply the sigmoid: $A_l = \sigma(Z_l) = \dfrac{1}{1 + e^{-Z_l}}$
   - Store $A_l$ in the activations dictionary
3. Return all activations (we'll need them for backpropagation).

**Forward Propagation — General Form**

For a network with $L$ layers:

$$A_0 = X \tag{1}$$
$$Z_l = W_l \cdot A_{l-1} + b_l \quad \text{for } l = 1, \ldots, L \tag{2}$$
$$A_l = \sigma(Z_l) \quad \text{for } l = 1, \ldots, L \tag{3}$$

The output prediction is $\hat{y} = A_L$.
Two operations per layer. $L$ times. That's it.

Note the key insight: we store **all** activations $A_0, A_1, \ldots, A_L$ in a dictionary. We'll need every single one of them during backpropagation — because each gradient depends on the activation of the previous layer.

## 5 Generalized Backpropagation

This is the heart of the episode.

In Episode VI, backpropagation was six specific lines — each gradient written out explicitly for layers 1 and 2. Now we need to generalize it to **any number of layers**.

The good news: the math doesn't change. The chain rule still works the same way. We just apply it $L$ times instead of twice.

## 5.1 The Equations

**Generalized Backpropagation Equations**

For the **output layer** ($l = L$):

$$dZ_L = A_L - y$$

For any **hidden layer** ($l = L-1,\ L-2,\ \ldots,\ 1$):

$$dZ_l = \left(W_{l+1}^T \cdot dZ_{l+1}\right) \odot A_l \odot (1 - A_l)$$

For **every layer** ($l = L,\ L-1,\ \ldots,\ 1$):

$$dW_l = \frac{1}{m} \cdot dZ_l \cdot A_{l-1}^T \tag{4}$$

$$db_l = \frac{1}{m} \sum dZ_l \tag{5}$$

Let's make sure you understand what each term means:

- $dZ_L = A_L - y$ — the error at the output. This is **identical** to Episode VI.
- $dZ_l = (W_{l+1}^T \cdot dZ_{l+1}) \odot A_l \odot (1 - A_l)$ — the error **propagated backward** from layer $l+1$ to layer $l$. The term $A_l \odot (1 - A_l)$ is the derivative of the sigmoid activation.
- $dW_l$ and $db_l$ — the gradients with respect to the weights and biases at layer $l$. These are computed the same way at every layer.

**The Pattern**

At each layer, the same three things happen:

1. Receive the error from the layer above ($dZ$)
2. Compute the gradients for this layer ($dW$, $db$)
3. Pass the error backward to the layer below

This is why it's called **back**propagation — the error flows backward through the network.

## 5.2 The Code

Generalized Backpropagation

```python
def back_propagation(y, parameters, activations):
    m = y.shape[1]
    L = len(parameters) // 2

    # Output layer error
    dZ = activations[f'A{L}'] - y
    gradients = {}

    # Loop backward through the layers
```

```
    for l in reversed(range(1, L + 1)):
        gradients[f'dW{l}'] = 1/m * dZ @ activations[f'A{l-1}'].T
        gradients[f'db{l}'] = 1/m * np.sum(dZ, axis=1, keepdims=True)

        # If not the first layer, propagate error backward
        if l > 1:
            dA_prev = parameters[f'W{l}'].T @ dZ
            A_prev = activations[f'A{l-1}']
            dZ = dA_prev * A_prev * (1 - A_prev)

    return gradients
```

Compare this to Episode VI's six hardcoded lines:

Episode VI — Hardcoded for 2 layers

```
dZ2 = A2 - y
dW2 = 1/m * dZ2 @ A1.T
db2 = 1/m * np.sum(dZ2, axis=1, keepdims=True)
dZ1 = W2.T @ dZ2 * A1 * (1 - A1)
dW1 = 1/m * dZ1 @ X.T
db1 = 1/m * np.sum(dZ1, axis=1, keepdims=True)
```

**Same Math — Just a Loop**

The generalized version does **exactly** the same thing. It starts at the output ($l = L$), computes $dW_L$ and $db_L$, then propagates the error backward to $l = L - 1$, and repeats. If $L = 2$, the loop runs twice — and produces the exact same six gradients as Episode VI. If $L = 10$, it runs ten times. **No code changes.**

# 6 Update and Predict

These two functions are almost trivially simple.

## 6.1 Update — Gradient Descent for All Layers

```
def update(gradients, parameters, learning_rate):
    L = len(parameters) // 2
    for l in range(1, L + 1):
        parameters[f'W{l}'] -= learning_rate * gradients[f'dW{l}']
        parameters[f'b{l}'] -= learning_rate * gradients[f'db{l}']
    return parameters
```

Same gradient descent rule as before: $W \leftarrow W - \alpha \cdot dW$. Applied to every layer in a loop.

## 6.2 Predict — Identical to Episode VI

```
def predict(X, parameters):
    activations = forward_propagation(X, parameters)
```

```
    L = len(parameters) // 2
    A = activations[f'A{L}']
    return A >= 0.5
```

Run forward propagation, take the last activation $A_L$, threshold at 0.5. Nothing changes.

**Checkpoint — Your Generalized Toolkit**

| Function | Ep. VI | Ep. VII |
|---|---|---|
| `initialisation` | 3 args $(n_0, n_1, n_2)$ | 1 list `dimensions` |
| `forward_propagation` | 4 hardcoded lines | loop over $L$ layers |
| `back_propagation` | 6 hardcoded lines | backward loop |
| `update` | 4 subtractions | loop over $L$ layers |
| `predict` | same | same |

Five functions. Same logic as before. Now generalized to any depth $L$.

## 7 The Training Loop

The training loop is structurally **identical** to Episode VI:

```
def neural_network(X, y, dimensions, learning_rate, epoch):
    parameters = initialisation(dimensions)
    train_loss = []
    train_acc = []

    for i in tqdm(range(epoch)):
        L = len(parameters) // 2
        activations = forward_propagation(X, parameters)

        # Track metrics
        train_loss.append(log_loss(y.flatten(),
                          activations[f'A{L}'].flatten()))
        predictions = predict(X, parameters)
        train_acc.append(accuracy_score(y.flatten(),
                         predictions.flatten()))

        # Learn
        gradients = back_propagation(y, parameters, activations)
        parameters = update(gradients, parameters, learning_rate)

    # Plot
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(train_loss, label='Train Loss')
    plt.title('Learning Curve')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.subplot(1, 2, 2)
    plt.plot(train_acc, label='Train Accuracy')
```

```
    plt.title('Accuracy Curve')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.show()

    return parameters
```

The only difference from Episode VI: the first argument to `initialisation` is now a `dimensions` list instead of three separate numbers.

**Same Structure, Infinite Power**

Your 2-layer network from Episode VI is a **special case** of this general network. Call `neural_network(X, y, [2, 32, 1], ...)` and you get exactly the same network as before. Call it with `[2, 64, 64, 64, 1]` and you get a 4-layer network. **Same function.**

# 8 The Testing Ground

Time to see what depth can do. We'll test on datasets of **increasing difficulty** — from simple circles to spirals — and observe how the network's behavior changes with architecture.

## 8.1 Circles — The Baseline

We start where Episode VI left off:

```
from sklearn.datasets import make_circles
X, y = make_circles(n_samples=100, noise=0.1, factor=0.3)
X = X.T
y = y.reshape((1, y.shape[0]))

parameters = neural_network(X, y, [2, 32, 32, 1],
                            learning_rate=0.1, epoch=500)
```

Result: ~100% accuracy. The network separates the two circles perfectly — just like in Episode VI, but now with an extra hidden layer.

## 8.2 Moons — Still Easy

```
from sklearn.datasets import make_moons
X, y = make_moons(n_samples=1000, noise=0.15)
X = X.T
y = y.reshape((1, y.shape[0]))

parameters = neural_network(X, y, [2, 64, 64, 1],
                            learning_rate=0.1, epoch=5000)
```

The moon-shaped data is harder than circles, but two hidden layers of 64 neurons handle it comfortably. The decision boundary curves smoothly around each crescent.

### 8.3 Spirals — The Real Test

Now we push the network to its limits:

```
parameters = neural_network(X, y, [2, 64, 64, 64, 1],
                            learning_rate=0.1, epoch=5000)
```

Spirals are interleaved — the two classes wrap around each other. This is a genuinely hard problem. With three hidden layers, the network reaches around 85% accuracy. Not perfect, but remember: this is pure NumPy. No frameworks. No tricks.

What if we go deeper?

```
parameters = neural_network(X, y, [2, 128, 128, 128, 128, 1],
                            learning_rate=0.1, epoch=5000)
```

Better. More layers, more neurons — the decision boundary becomes more complex, wrapping tighter around the spiral arms.

**Depth vs Complexity**

More complex data requires **more layers**. That's the whole point of depth.

- **Circles**: 2 layers suffice (simple nonlinearity)
- **Moons**: 2 layers work, 3 are comfortable
- **Spirals**: 3–4 layers needed (highly nonlinear boundary)

Each layer adds another level of **abstraction** — the ability to capture more complex patterns.

## 9 Real Data — Cats vs Dogs, Going Deeper

Let's go back to the real-world challenge from Episode VI: classifying images of cats and dogs.

The data: $64 \times 64$ pixel images, flattened to 4096-dimensional vectors. 1000 training images, a few hundred for testing.

In Episode VI, we used the equivalent of `[4096, 32, 32, 1]`. Now we can try deeper architectures. But first, we need a version of the training loop that tracks **both** training and test performance:

```
def neural_network2(X_train, y_train, X_test, y_test,
                    dimensions, learning_rate, epoch):
    parameters = initialisation(dimensions)
    train_loss, train_acc = [], []
    test_loss, test_acc = [], []

    for i in tqdm(range(epoch)):
        L = len(parameters) // 2
        activations = forward_propagation(X_train, parameters)

        if i % 100 == 0:
```

```
            # Training metrics
            train_loss.append(log_loss(y_train.flatten(),
                                       activations[f'A{L}'].flatten()))
            predictions = predict(X_train, parameters)
            train_acc.append(accuracy_score(y_train.flatten(),
                                            predictions.flatten()))

            # Test metrics
            act_test = forward_propagation(X_test, parameters)
            test_loss.append(log_loss(y_test.flatten(),
                                      act_test[f'A{L}'].flatten()))
            pred_test = predict(X_test, parameters)
            test_acc.append(accuracy_score(y_test.flatten(),
                                           pred_test.flatten()))

        gradients = back_propagation(y_train, parameters,
                                     activations)
        parameters = update(gradients, parameters, learning_rate)

    # Plot train vs test
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(train_loss, label='Train Loss')
    plt.plot(test_loss, label='Test Loss')
    plt.legend()
    plt.title('Loss')
    plt.subplot(1, 2, 2)
    plt.plot(train_acc, label='Train Accuracy')
    plt.plot(test_acc, label='Test Accuracy')
    plt.legend()
    plt.title('Accuracy')
    plt.show()

    return parameters
```

Now let's try:

```
parameters = neural_network2(
    X_train, y_train, X_test, y_test,
    [X_train.shape[0], 16, 16, 1],
    learning_rate=0.01, epoch=5000
)
```

The result will look familiar: training accuracy climbs toward 90–95%, but test accuracy plateaus around 50–55%.

*The same overfitting wall we hit in Episode VI.*

## 10 The Overfitting Wall

Here's the uncomfortable truth: adding more layers **doesn't solve** the overfitting problem.

Try `[4096, 64, 32, 16, 1]`. Try `[4096, 128, 64, 32, 16, 1]`. The training accuracy might reach 99%. But the test accuracy? Still stuck around 50–55%.

**More Depth ≠ Better Generalization**

More layers give the network more **capacity** — the ability to learn more complex patterns. But capacity without discipline is dangerous.
A deeper network can memorize the training data more efficiently. It doesn't mean it **understands** the data.

This is the same lesson from Episode VI, but now it's even more striking. We can build a network of *any depth*, and it still can't break the 55% barrier on test data.

Why?

1. **Not enough data.** 1000 images isn't enough to train a network with thousands of parameters.
2. **No regularization.** We have no mechanism to prevent the network from overfitting.
3. **Optimization challenges.** Deep networks with sigmoid activations can suffer from vanishing gradients — the error signal becomes too small to update the early layers effectively.

**The Million-Dollar Question**

When your model fails, always ask:

> **"Is this a problem with the data, the model, or the training?"**

Here, it's **all three**:

- **Data**: not enough images for this task
- **Model**: sigmoid activation causes vanishing gradients in deep networks
- **Training**: no regularization, no dropout, no data augmentation

Each of these has a solution. But that's the next chapter.

## 11 What You've Built

Take a step back and look at what just happened.

In Episode VI, you coded a neural network with two layers. Today, you **removed the constraint**. You generalized five functions — and now you can build a network of **any depth**.

**Your Deep Neural Network — From Scratch**

1. **Architecture definition**: a single `dimensions` list
2. **Initialization**: one loop creates all $W_l$ and $b_l$
3. **Forward propagation**: one loop computes all $A_l$
4. **Backpropagation**: one backward loop computes all gradients
5. **Update**: one loop applies gradient descent to all parameters
6. **Training loop**: identical structure, works for any depth
7. **Testing**: circles, moons, spirals, cats vs dogs

The code for a 2-layer network and a 100-layer network is **the same**. Only the dimensions list changes.

That's the power of generalization.

*You don't need a bigger codebase. You need a smarter one.*

## 12 What's Next

You now have the power to build deep networks. But power without discipline leads to overfitting, vanishing gradients, and wasted computation.

The next step isn't more layers. It's learning how to **control** what we've built.

**Next Stop — Discipline**

We need to teach the network discipline:

- **Better activations**: ReLU instead of sigmoid
- **Regularization**: preventing memorization
- **Better optimization**: beyond basic gradient descent
- **More data**: or smarter use of what we have

Depth is the foundation. Discipline is what makes it work.
**Meet me in the next episode.**

---

**Deep Learning From Scratch**

An open-source learning journey.

**GitHub:** https://github.com/Pchambet/Deep-Learning-from-Scratch

**LinkedIn:** https://www.linkedin.com/in/pierre-chambet/

© 2026 Pierre Chambet